

ESTIA Documentation V1.0

Ανάπτυξη της εφαρμογής ESTIA – AI Concierge Assistant για Ξενοδοχειακές Μονάδες

Τεχνική Περιγραφή και Υλοποίηση

Contents

Πρόλογος.....	4
1. Εισαγωγή.....	5
2. Στόχος της εφαρμογής	5
3. Τεχνολογίες που χρησιμοποίησα	6
3.1 Python	6
3.2 FastAPI.....	6
3.3 OpenAI GPT-4o	7
3.4 ChromaDB	7
3.5 Markdown	7
3.6 HTML, CSS και JavaScript.....	8
3.7 Git και GitHub.....	8
4. Δομή του Project	8
4.1 app.....	9
4.2 knowledge	9
4.3 static.....	10
4.4 tests	10
5. Πώς λειτουργεί η ESTIA.....	11
5.1 Ο χρήστης υποβάλλει μία ερώτηση	12
5.2 Το FastAPI λαμβάνει το αίτημα.....	12
5.3 Αναγνώριση γλώσσας	12
5.4 Guest Context Memory.....	13
5.5 Intent Detection	13
5.6 Αναζήτηση στη βάση γνώσης (RAG).....	13
5.7 Δημιουργία της απάντησης.....	14
5.8 Επιστροφή της απάντησης	14

6. Η βάση γνώσης (Knowledge Base).....	14
7. Guest Context Memory	15
7.1 Ποιες πληροφορίες αποθηκεύονται.....	15
7.2 Παράδειγμα λειτουργίας.....	16
7.3 Γιατί δεν χρησιμοποιείται μόνο το ιστορικό συνομιλίας.....	16
7.4 Πλεονεκτήματα	17
8. Grounded Retrieval.....	17
8.1 Τι σημαίνει Grounded.....	17
8.2 Η διαδικασία.....	17
8.3 Παράδειγμα.....	18
8.4 Grounding Rules	18
8.5 Ένα πραγματικό παράδειγμα από την ανάπτυξη	18
9. Retrieval-Augmented Generation (RAG)	19
10. Δομή του κώδικα της εφαρμογής.....	19
10.1 Ο φάκελος app	20
10.2 Ο φάκελος api.....	20
10.3 Ο φάκελος services	21
10.4 Ο φάκελος models.....	23
10.5 Ο φάκελος core.....	24
10.6 Ο φάκελος scripts	24
10.7 Ο φάκελος utils	24
11. Πώς συνεργάζονται όλα μεταξύ τους.....	24
12. Οι δυσκολίες που αντιμετωπίσα κατά την ανάπτυξη.....	26
12.1 Hallucinations	26
12.2 Η εφαρμογή δεν θυμόταν τον επισκέπτη.....	26
12.3 Μη σχετικές απαντήσεις.....	27
12.4 Grounding Rules.....	27
13. Δοκιμές της εφαρμογής.....	27
13.1 Unit Tests	28
13.2 Integration Tests.....	28
13.3 Πραγματικά σενάρια.....	28
14. Αποτελέσματα.....	29
15. Μελλοντικές Δυνατότητες (Future Improvements).....	29

15.1 Διασύνδεση με Opera PMS.....	30
15.2 Online Κρατήσεις.....	30
15.3 Φωνητική Συνομιλία	30
15.4 Mobile Application	31
15.5 Υποστήριξη περισσότερων ξενοδοχείων.....	31
15.6 Διασύνδεση με CRM.....	31
15.7 Συνεχής Ενημέρωση της Βάσης Γνώσης.....	31
15.8 Προσωπική Αξιολόγηση.....	31
16. Τι έμαθα μέσα από αυτή την εργασία	31
17. Συμπεράσματα	32
Παράρτημα.....	33
Παράρτημα Α – Δομή του Project	34
estia-ai-concierge/	34
└─ app/.....	34
└─ api/.....	34
└─ core/.....	34
└─ models/.....	34
└─ services/	34
└─ scripts/	34
└─ utils/.....	34
└─ knowledge/.....	34
└─ static/	34
└─ tests/	34
└─ docs/	34
└─ screenshots/.....	34
└─ Dockerfile	34
└─ docker-compose.yml.....	34
└─ requirements.txt.....	34
└─ README.md	34
└─ .env	34
Παράρτημα Β – Βασικά Endpoints.....	35
Παράρτημα Γ – Βασικές Τεχνολογίες.....	35
Παράρτημα Δ – GitHub.....	36

Πρόλογος

Η εργασία αυτή παρουσιάζει την ανάπτυξη της εφαρμογής **ESTIA**, ενός βοηθού τεχνητής νοημοσύνης (AI Concierge Assistant) που σχεδιάστηκε για χρήση σε ξενοδοχειακές μονάδες πολυτελείας.

Η ιδέα γεννήθηκε από μια πραγματική ανάγκη που αντιμετωπίζουν καθημερινά τα ξενοδοχεία. Οι επισκέπτες ζητούν συνεχώς πληροφορίες σχετικά με τα εστιατόρια, το Spa, τις μεταφορές, τις δραστηριότητες, τις εκδρομές και πολλές ακόμη υπηρεσίες. Αν και οι πληροφορίες αυτές υπάρχουν, συνήθως είναι διάσπαρτες σε έγγραφα ή είναι γνωστές μόνο από το προσωπικό του ξενοδοχείου.

Στόχος μου ήταν να δημιουργήσω μία εφαρμογή που θα μπορεί να απαντά σε αυτές τις ερωτήσεις χρησιμοποιώντας τεχνητή νοημοσύνη, αλλά χωρίς να δίνει τυχαίες ή λανθασμένες πληροφορίες.

Κατά τη διάρκεια της ανάπτυξης ήρθα σε επαφή με τεχνολογίες όπως η Python, η FastAPI, τα Large Language Models, το Retrieval-Augmented Generation (RAG) και οι Vector Databases. Μέσα από το συγκεκριμένο έργο κατάφερα να συνδυάσω όλες αυτές τις τεχνολογίες σε μία ολοκληρωμένη εφαρμογή.

Η εργασία αυτή περιγράφει τόσο τη διαδικασία σχεδιασμού όσο και την τεχνική υλοποίηση της ESTIA, εξηγώντας με απλό τρόπο τις επιλογές που έγιναν και τα αποτελέσματα που προέκυψαν.

1. Εισαγωγή

Η τεχνητή νοημοσύνη τα τελευταία χρόνια χρησιμοποιείται ολοένα και περισσότερο στον χώρο του τουρισμού και της φιλοξενίας. Οι σύγχρονοι επισκέπτες επιθυμούν να λαμβάνουν γρήγορες και προσωποποιημένες απαντήσεις οποιαδήποτε στιγμή της ημέρας, χωρίς να χρειάζεται πάντα η παρέμβαση ενός υπαλλήλου.

Παρόλο που υπάρχουν ήδη αρκετά chatbot, τα περισσότερα λειτουργούν είτε με προκαθορισμένους κανόνες είτε βασίζονται αποκλειστικά σε μεγάλα γλωσσικά μοντέλα (LLMs). Στη δεύτερη περίπτωση υπάρχει ο κίνδυνος το μοντέλο να δημιουργήσει πληροφορίες που δεν είναι αληθινές. Το φαινόμενο αυτό είναι γνωστό ως **hallucination**.

Για τον λόγο αυτό αποφάσισα να δημιουργήσω την εφαρμογή **ESTIA**, η οποία πριν απαντήσει σε μία ερώτηση αναζητά πρώτα πληροφορίες στη δική της βάση γνώσης και μόνο τότε δημιουργεί την τελική απάντηση.

Με αυτόν τον τρόπο οι απαντήσεις είναι περισσότερο αξιόπιστες και βασίζονται στις πραγματικές πληροφορίες του ξενοδοχείου.

2. Στόχος της εφαρμογής

Ο βασικός στόχος της ESTIA είναι να λειτουργεί ως ένας ψηφιακός concierge για ξενοδοχεία πολυτελείας.

Η εφαρμογή μπορεί να ενημερώνει τον επισκέπτη σχετικά με:

- τα εστιατόρια του ξενοδοχείου,
- τα bars,
- το Spa,
- τις μεταφορές από και προς το αεροδρόμιο,
- τις θαλάσσιες εκδρομές,
- τις υπηρεσίες VIP,
- τις δραστηριότητες για οικογένειες,
- τις αθλητικές δραστηριότητες,
- τις υπηρεσίες των διαφορετικών ξενοδοχειακών μονάδων.

Εκτός όμως από την απλή παροχή πληροφοριών, ένας ακόμη στόχος ήταν η εφαρμογή να μπορεί να **θυμάται** βασικές πληροφορίες που αναφέρει ο επισκέπτης κατά τη διάρκεια της συνομιλίας.

Για παράδειγμα, αν ένας επισκέπτης αναφέρει ότι διαμένει στο Porto Elounda και ταξιδεύει με δύο παιδιά, η εφαρμογή θα χρησιμοποιήσει αυτή την πληροφορία και στις επόμενες απαντήσεις χωρίς να χρειάζεται να την επαναλάβει.

Η δυνατότητα αυτή κάνει τη συνομιλία πιο φυσική και πιο κοντά στην εμπειρία που θα είχε ένας επισκέπτης με έναν πραγματικό concierge.

3. Τεχνολογίες που χρησιμοποίησα

Για την ανάπτυξη της εφαρμογής ESTIA χρησιμοποίησα τεχνολογίες που γνώρισα κατά τη διάρκεια του προγράμματος AI Developer, αλλά και μέσα από προσωπική μελέτη και πειραματισμό. Προσπάθησα να επιλέξω εργαλεία που είναι σύγχρονα, αξιόπιστα και κατάλληλα για εφαρμογές Τεχνητής Νοημοσύνης.

Στις επόμενες ενότητες παρουσιάζω συνοπτικά κάθε τεχνολογία και τον λόγο που την επέλεξα.

3.1 Python

Η βασική γλώσσα προγραμματισμού που χρησιμοποίησα είναι η Python.

Η Python θεωρείται μία από τις πιο δημοφιλείς γλώσσες για εφαρμογές Τεχνητής Νοημοσύνης, Machine Learning και Data Science. Ένας από τους λόγους είναι ότι διαθέτει μεγάλο αριθμό βιβλιοθηκών, ενώ ο κώδικάς της είναι απλός και εύκολος στην κατανόηση.

Επέλεξα την Python γιατί:

- είναι η βασική γλώσσα του προγράμματος AI Developer,
- συνεργάζεται εύκολα με υπηρεσίες Τεχνητής Νοημοσύνης,
- διαθέτει πολλές βιβλιοθήκες για ανάπτυξη εφαρμογών,
- μου επέτρεψε να δημιουργήσω την εφαρμογή χωρίς να χρειαστεί να χρησιμοποιήσω πολλές διαφορετικές γλώσσες προγραμματισμού.

Όλη η λογική της εφαρμογής ESTIA έχει υλοποιηθεί σε Python.

3.2 FastAPI

Για το backend της εφαρμογής επέλεξα να χρησιμοποιήσω το FastAPI. Το FastAPI είναι ένα σύγχρονο framework για ανάπτυξη REST API στην Python. Ο ρόλος του είναι να δέχεται τα αιτήματα του χρήστη, να τα επεξεργάζεται και να επιστρέφει τις απαντήσεις.

Στην εφαρμογή ESTIA το FastAPI χρησιμοποιείται για:

- επικοινωνία με το frontend,

- επικοινωνία με το OpenAI,
- ανάκτηση πληροφοριών από τη βάση γνώσης,
- διαχείριση της συνομιλίας,
- έλεγχο της κατάστασης της εφαρμογής (health endpoint).

Η επιλογή του FastAPI έγινε επειδή είναι γρήγορο, εύκολο στη χρήση και κατάλληλο για εφαρμογές AI.

3.3 OpenAI GPT-4o

Η ESTIA χρησιμοποιεί το μοντέλο GPT-4o της OpenAI για τη δημιουργία των απαντήσεων.

Το GPT-4o δεν διαθέτει από μόνο του πληροφορίες για το συγκεκριμένο ξενοδοχείο. Για τον λόγο αυτό δεν του επιτρέπεται να απαντά μόνο με βάση τις γενικές του γνώσεις. Πριν δημιουργήσει μία απάντηση, λαμβάνει πληροφορίες από τη βάση γνώσης της εφαρμογής.

Με αυτόν τον τρόπο οι απαντήσεις βασίζονται στα πραγματικά δεδομένα του ξενοδοχείου.

3.4 ChromaDB

Για την αποθήκευση της βάσης γνώσης χρησιμοποιείται η ChromaDB. Η ChromaDB είναι μία βάση δεδομένων διανυσμάτων (Vector Database). Σε αντίθεση με μία κλασική βάση δεδομένων, η ChromaDB δεν αναζητά μόνο ίδιες λέξεις αλλά και παρόμοιο νόημα.

Έτσι, ακόμη και αν ο επισκέπτης διατυπώσει διαφορετικά την ερώτησή του, η εφαρμογή μπορεί να εντοπίσει το κατάλληλο έγγραφο.

Η ChromaDB αποτελεί βασικό κομμάτι της αρχιτεκτονικής RAG.

3.5 Markdown

Όλες οι πληροφορίες του ξενοδοχείου αποθηκεύονται σε αρχεία Markdown (.md). Αρχικά σκέφτηκα να χρησιμοποιήσω μία βάση δεδομένων. Στη συνέχεια όμως διαπίστωσα ότι τα αρχεία Markdown είναι πολύ πιο εύκολα στη διαχείριση. Με αυτόν τον τρόπο μπορεί κάποιος να ενημερώνει τις πληροφορίες του ξενοδοχείου χωρίς να γνωρίζει προγραμματισμό.

Για παράδειγμα, μπορεί να αλλάξει:

- το ωράριο ενός εστιατορίου,
- τις υπηρεσίες του Spa,
- μία νέα δραστηριότητα,
- μία νέα εκδρομή,

απλώς επεξεργαζόμενος ένα αρχείο κειμένου.

3.6 HTML, CSS και JavaScript

Για το γραφικό περιβάλλον της εφαρμογής χρησιμοποίησα:

- HTML
- CSS
- JavaScript

Η HTML χρησιμοποιήθηκε για τη δημιουργία της σελίδας.

Η CSS χρησιμοποιήθηκε για τη μορφοποίηση της εφαρμογής.

Η JavaScript αναλαμβάνει την επικοινωνία με το backend και την εμφάνιση των απαντήσεων στον χρήστη.

Παρότι το frontend είναι σχετικά απλό, καλύπτει πλήρως τις ανάγκες της εφαρμογής.

3.7 Git και GitHub

Κατά τη διάρκεια της ανάπτυξης χρησιμοποίησα το Git για την παρακολούθηση των αλλαγών στον κώδικα.

Το GitHub χρησιμοποιήθηκε για:

- αποθήκευση του έργου,
- δημιουργία εκδόσεων (versions),
- ιστορικό αλλαγών,
- παρουσίαση της εφαρμογής.

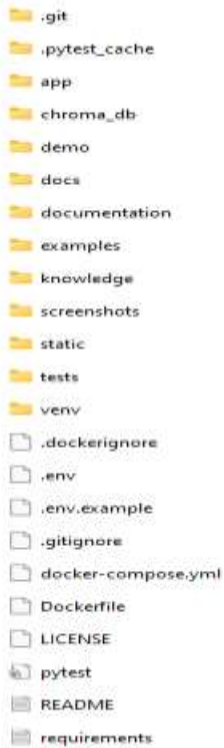
Η χρήση του GitHub με βοήθησε να οργανώσω καλύτερα την ανάπτυξη της εφαρμογής και να διατηρώ αντίγραφα ασφαλείας του έργου.

4. Δομή του Project

Από την αρχή προσπάθησα να οργανώσω τον κώδικα σε φακέλους ώστε να είναι εύκολος στη συντήρηση και στην επέκταση.

Η βασική δομή του έργου είναι η παρακάτω:

estia-ai-concierge/



κάθε φάκελος έχει διαφορετικό ρόλο.

4.1 app

Ο φάκελος **app** περιέχει όλο τον βασικό κώδικα της εφαρμογής.

Εδώ βρίσκονται:

- τα API,
- τα services,
- τα μοντέλα δεδομένων,
- οι βοηθητικές συναρτήσεις.

Μπορούμε να πούμε ότι αποτελεί την «καρδιά» της ESTIA.

4.2 knowledge

Ο φάκελος knowledge περιέχει όλες τις πληροφορίες του ξενοδοχείου. Οι πληροφορίες είναι οργανωμένες σε κατηγορίες όπως:

- Restaurants
- Spa
- Bars

- Family
- Transportation
- Premium Services
- Sports
- FAQs

Κάθε κατηγορία αποτελείται από αρχεία Markdown.

Αυτή η οργάνωση κάνει πολύ εύκολη την προσθήκη νέας γνώσης χωρίς να αλλάζει ο κώδικας της εφαρμογής.

4.3 static

Ο φάκελος **static** περιέχει το γραφικό περιβάλλον της εφαρμογής.

Εδώ βρίσκονται:

- η αρχική σελίδα,
- τα αρχεία CSS,
- τα αρχεία JavaScript.

Όταν ο χρήστης ανοίγει την εφαρμογή από τον browser, φορτώνονται τα αρχεία αυτού του φακέλου.

4.4 tests

Καθ' όλη τη διάρκεια της ανάπτυξης δημιούργησα αρκετά tests ώστε να ελέγχω ότι οι αλλαγές που πραγματοποιούσα δεν δημιουργούσαν νέα προβλήματα.

Τα tests ελέγχουν μεταξύ άλλων:

- το Guest Context,
- το Retrieval,
- το Chat Service,
- το Conversation Store,
- το Intent Detection,
- το Grounded Retrieval.

Με αυτόν τον τρόπο μπορούσα να είμαι πιο σίγουρος ότι κάθε νέα λειτουργία συνέχιζε να λειτουργεί σωστά.

5. Πώς λειτουργεί η ESTIA

Σε αυτό το κεφάλαιο παρουσιάζω με απλά λόγια τη συνολική λειτουργία της εφαρμογής.

Από την πλευρά του χρήστη η διαδικασία φαίνεται αρκετά απλή. Ο επισκέπτης ανοίγει την εφαρμογή, γράφει την ερώτησή του και λαμβάνει την απάντηση μέσα σε λίγα δευτερόλεπτα.

Στην πραγματικότητα όμως, μέχρι να εμφανιστεί αυτή η απάντηση, πραγματοποιούνται αρκετά βήματα στο παρασκήνιο.

Η παρακάτω εικόνα παρουσιάζει συνοπτικά τη ροή της εφαρμογής.

Χρήστης



Frontend (HTML / JavaScript)



FastAPI Backend



Ανίχνευση Γλώσσας



Guest Context Memory

|
▼
Intent Detection

|
▼
RAG Retrieval

|
▼
OpenAI GPT-4o

|
▼
Απάντηση στον χρήστη

Παρακάτω περιγράφω αναλυτικά κάθε στάδιο.

5.1 Ο χρήστης υποβάλλει μία ερώτηση

Όλα ξεκινούν όταν ο επισκέπτης πληκτρολογήσει μία ερώτηση.

Παραδείγματα:

- «Πού μπορώ να φάω απόψε;»
- «Έχετε Spa;»
- «Θέλω μεταφορά από το αεροδρόμιο.»
- «Έχουμε δύο παιδιά. Τι δραστηριότητες υπάρχουν;»

Η ερώτηση αποστέλλεται από το frontend προς το backend μέσω ενός REST API.

5.2 Το FastAPI λαμβάνει το αίτημα

Το FastAPI είναι το πρώτο σημείο επεξεργασίας της ερώτησης. Εδώ πραγματοποιούνται οι βασικοί έλεγχοι και ξεκινά η διαδικασία δημιουργίας της απάντησης.

Ο backend αναλαμβάνει να συντονίσει όλα τα υπόλοιπα τμήματα της εφαρμογής.

5.3 Αναγνώριση γλώσσας

Η εφαρμογή υποστηρίζει τόσο Ελληνικά όσο και Αγγλικά. Για τον λόγο αυτό γίνεται αρχικά προσπάθεια να αναγνωριστεί η γλώσσα στην οποία έγραψε ο επισκέπτης. Έτσι η ESTIA

μπορεί να απαντήσει στην ίδια γλώσσα χωρίς να χρειάζεται ο χρήστης να επιλέξει γλώσσα από κάποιο μενού.

5.4 Guest Context Memory

Ένα από τα πιο ενδιαφέροντα χαρακτηριστικά της εφαρμογής είναι ότι θυμάται πληροφορίες που αναφέρει ο επισκέπτης.

Για παράδειγμα:

We are staying at Porto Elounda.

Η εφαρμογή αποθηκεύει ότι ο επισκέπτης βρίσκεται στο Porto Elounda.

Αν στη συνέχεια γράψει:

Where should we have dinner?

η ESTIA γνωρίζει ήδη σε ποιο ξενοδοχείο διαμένει και προτείνει τα κατάλληλα εστιατόρια.

Το ίδιο συμβαίνει και όταν αναφερθούν παιδιά, ειδικές διατροφικές ανάγκες ή κάποια ιδιαίτερη περίπτωση, όπως ταξίδι του μέλιτος.

Με αυτόν τον τρόπο η συνομιλία γίνεται πιο φυσική.

5.5 Intent Detection

Στη συνέχεια η εφαρμογή προσπαθεί να καταλάβει το θέμα της ερώτησης. Δεν αρκεί να γνωρίζει μόνο το κείμενο. Πρέπει να γνωρίζει και σε ποια κατηγορία ανήκει.

Για παράδειγμα:

Ερώτηση	Κατηγορία
Where should we have dinner?	Restaurant
I would like a massage	Spa
We need a taxi	Transportation
Can we charter a yacht?	Premium
Activities for children	Family

Η πληροφορία αυτή βοηθά την εφαρμογή να αναζητήσει πιο σωστά δεδομένα στη βάση γνώσης.

5.6 Αναζήτηση στη βάση γνώσης (RAG)

Αφού προσδιοριστεί η κατηγορία της ερώτησης, η εφαρμογή πραγματοποιεί αναζήτηση στη βάση γνώσης.

Η βάση γνώσης περιέχει όλες τις πληροφορίες που έχω καταχωρήσει σχετικά με:

- τα ξενοδοχεία,
- τα εστιατόρια,
- το Spa,
- τις μεταφορές,
- τα yacht charters,
- τις δραστηριότητες,
- τις VIP υπηρεσίες.

Η εφαρμογή δεν στέλνει αμέσως την ερώτηση στο GPT. Πρώτα βρίσκει τα πιο σχετικά έγγραφα.

Αυτό είναι το βασικό χαρακτηριστικό της τεχνικής Retrieval-Augmented Generation (RAG).

5.7 Δημιουργία της απάντησης

Μόλις βρεθούν οι σωστές πληροφορίες, αποστέλλονται μαζί με την ερώτηση στο μοντέλο GPT-4o. Το μοντέλο δεν απαντά χρησιμοποιώντας μόνο τις γενικές του γνώσεις. Αντίθετα, χρησιμοποιεί τις πληροφορίες που έλαβε από τη βάση γνώσης.

Έτσι οι απαντήσεις είναι πιο σωστές και πιο σχετικές με το συγκεκριμένο ξενοδοχείο.

5.8 Επιστροφή της απάντησης

Η τελική απάντηση επιστρέφει στο backend και από εκεί στο frontend.

Ο επισκέπτης βλέπει την απάντηση μέσα στο παράθυρο συνομιλίας χωρίς να αντιλαμβάνεται όλα τα βήματα που προηγήθηκαν.

6. Η βάση γνώσης (Knowledge Base)

Η βάση γνώσης αποτελεί ίσως το σημαντικότερο μέρος της εφαρμογής. Αντί να αποθηκεύσω τις πληροφορίες μέσα στον κώδικα, αποφάσισα να δημιουργήσω έναν ξεχωριστό φάκελο με αρχεία Markdown. Η επιλογή αυτή είχε αρκετά πλεονεκτήματα.

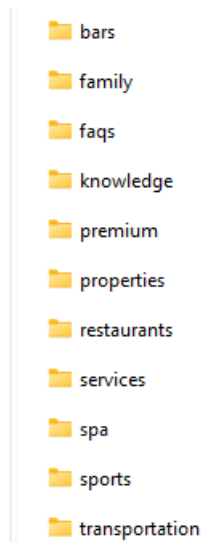
Πρώτον, οι πληροφορίες μπορούν να ενημερώνονται χωρίς να αλλάζει ο κώδικας.

Δεύτερον, η δομή παραμένει οργανωμένη.

Τρίτον, είναι πολύ εύκολο να προστεθούν νέες υπηρεσίες ή νέα ξενοδοχεία.

Η βασική δομή της γνώσης είναι η παρακάτω:

knowledge/



Κάθε φάκελος περιέχει τις πληροφορίες μιας συγκεκριμένης κατηγορίας.

Για παράδειγμα, στον φάκελο **restaurants** υπάρχουν ξεχωριστά αρχεία για κάθε εστιατόριο. Με αυτόν τον τρόπο η εφαρμογή μπορεί να εντοπίσει μόνο τις πληροφορίες που σχετίζονται με την ερώτηση του επισκέπτη.

7. Guest Context Memory

Κατά τη διάρκεια της ανάπτυξης της εφαρμογής διαπίστωσα ότι ένας πραγματικός concierge δεν αντιμετωπίζει κάθε ερώτηση σαν να είναι η πρώτη. Αν ένας επισκέπτης πει ότι μένει στο Porto Elounda και έχει δύο παιδιά, ο concierge θα θυμάται αυτές τις πληροφορίες και θα τις χρησιμοποιεί σε ολόκληρη τη συζήτηση. Ήθελα η ESTIA να συμπεριφέρεται με τον ίδιο τρόπο.

Για τον λόγο αυτό δημιούργησα έναν μηχανισμό που ονόμασα **Guest Context Memory**. Ο μηχανισμός αυτός επιτρέπει στην εφαρμογή να αποθηκεύει βασικές πληροφορίες που αναφέρει ο επισκέπτης και να τις χρησιμοποιεί στις επόμενες απαντήσεις. Με αυτόν τον τρόπο η συνομιλία γίνεται πιο φυσική και δεν απαιτείται ο επισκέπτης να επαναλαμβάνει συνεχώς τις ίδιες πληροφορίες.

7.1 Ποιες πληροφορίες αποθηκεύονται

Η εφαρμογή δεν αποθηκεύει ολόκληρη τη συνομιλία ως μία μεγάλη παράγραφο. Αντίθετα, αποθηκεύει μόνο τις πληροφορίες που είναι πραγματικά χρήσιμες.

Στην τρέχουσα έκδοση της ESTIA αποθηκεύονται:

- το ξενοδοχείο στο οποίο διαμένει ο επισκέπτης,
- ο τύπος του επισκέπτη (οικογένεια, ζευγάρι, VIP κ.λπ.),

- η ηλικία των παιδιών,
- ειδικές περιστάσεις (π.χ. ταξίδι του μέλιτος),
- ενδιαφέροντα,
- διατροφικές προτιμήσεις,
- ανάγκες προσβασιμότητας.

Με αυτόν τον τρόπο η εφαρμογή μπορεί να προσαρμόζει τις προτάσεις της.

7.2 Παράδειγμα λειτουργίας

Παρακάτω φαίνεται ένα πραγματικό παράδειγμα.

Μήνυμα 1

We are staying at Porto Elounda.

Η ESTIA αποθηκεύει ότι ο επισκέπτης διαμένει στο Porto Elounda.

Μήνυμα 2

We have children aged 5 and 8.

Η εφαρμογή ενημερώνει τη μνήμη της.

Πλέον γνωρίζει ότι:

- πρόκειται για οικογένεια,
- υπάρχουν δύο παιδιά,
- οι ηλικίες είναι 5 και 8 ετών.

Μήνυμα 3

Where should we have dinner tonight?

Παρόλο που στην ερώτηση δεν αναφέρεται ξανά το ξενοδοχείο ή τα παιδιά, η εφαρμογή θυμάται ήδη αυτές τις πληροφορίες. Έτσι μπορεί να προτείνει εστιατόρια που είναι κατάλληλα για οικογένειες και βρίσκονται στο συγκεκριμένο ξενοδοχείο.

Αυτό ήταν ένα από τα σημαντικότερα χαρακτηριστικά που ήθελα να αποκτήσει η ESTIA.

7.3 Γιατί δεν χρησιμοποιείται μόνο το ιστορικό συνομιλίας

Αρχικά πίστευα ότι αρκεί να στέλνω στο μοντέλο όλη τη συνομιλία. Στην πράξη όμως διαπίστωσα ότι αυτό δεν είναι πάντα αρκετό.

Όσο μεγαλώνει η συνομιλία:

- αυξάνεται ο αριθμός των tokens,
- αυξάνεται το κόστος,
- μειώνεται η ποιότητα των απαντήσεων.

Για τον λόγο αυτό δημιούργησα ένα ξεχωριστό αντικείμενο μνήμης (Guest Context), στο οποίο αποθηκεύονται μόνο οι πληροφορίες που είναι πραγματικά χρήσιμες. Με αυτόν τον τρόπο η εφαρμογή παραμένει πιο αποδοτική.

7.4 Πλεονεκτήματα

Η χρήση του Guest Context Memory προσφέρει αρκετά πλεονεκτήματα.

- Οι συνομιλίες γίνονται πιο φυσικές.
- Ο επισκέπτης δεν χρειάζεται να επαναλαμβάνει συνεχώς τις ίδιες πληροφορίες.
- Οι προτάσεις της εφαρμογής γίνονται πιο προσωποποιημένες.
- Μειώνεται η πιθανότητα λανθασμένων προτάσεων.

Πιστεύω ότι αυτό είναι ένα από τα χαρακτηριστικά που διαφοροποιούν την ESTIA από ένα απλό chatbot.

8. Grounded Retrieval

Κατά τη διάρκεια των δοκιμών παρατήρησα ότι τα μεγάλα γλωσσικά μοντέλα πολλές φορές δημιουργούν πληροφορίες που δεν υπάρχουν στην πραγματικότητα. Για παράδειγμα, όταν ρώτησα για κάποιο εστιατόριο, το μοντέλο δημιούργησε ονόματα που δεν υπήρχαν στο ξενοδοχείο.

Το φαινόμενο αυτό ονομάζεται **Hallucination**.

Για μία εφαρμογή που προορίζεται για πραγματικούς επισκέπτες ξενοδοχείου, κάτι τέτοιο δεν είναι αποδεκτό. Για τον λόγο αυτό υλοποίησα μία αρχιτεκτονική **Grounded Retrieval**.

8.1 Τι σημαίνει Grounded

Η λέξη *Grounded* σημαίνει ότι κάθε απάντηση πρέπει να βασίζεται σε πραγματικά δεδομένα. Η εφαρμογή δεν πρέπει να απαντά χρησιμοποιώντας μόνο τις γενικές γνώσεις του μοντέλου. Πρέπει πρώτα να αναζητά πληροφορίες στη βάση γνώσης. Μόνο τότε δημιουργείται η τελική απάντηση.

8.2 Η διαδικασία

Η διαδικασία που ακολουθεί η ESTIA είναι η εξής.

1. Ο επισκέπτης υποβάλλει μία ερώτηση.
2. Η εφαρμογή αναγνωρίζει το θέμα της ερώτησης.

3. Αναζητά σχετικά έγγραφα στη βάση γνώσης.
4. Τα έγγραφα αποστέλλονται μαζί με την ερώτηση στο GPT-4o.
5. Το GPT δημιουργεί την απάντηση χρησιμοποιώντας μόνο τις πληροφορίες που βρέθηκαν.

Με αυτόν τον τρόπο μειώνεται σημαντικά η πιθανότητα εμφάνισης λανθασμένων πληροφοριών.

8.3 Παράδειγμα

Ερώτηση

Can I charter a yacht to Spinalonga?

Η εφαρμογή δεν απαντά αμέσως.

Πρώτα αναζητά το αρχείο:

knowledge/premium/yacht_charters.md

Από εκεί λαμβάνει τις διαθέσιμες πληροφορίες και δημιουργεί την απάντηση. Αν κάποια πληροφορία δεν υπάρχει στο αρχείο, η εφαρμογή δεν την επινοεί. Αντίθετα, προτείνει στον επισκέπτη να επικοινωνήσει με το Concierge.

8.4 Grounding Rules

Για να περιορίσω ακόμη περισσότερο τα hallucinations πρόσθεσα ειδικούς κανόνες (Grounding Rules) στο system prompt. Οι κανόνες αυτοί ενημερώνουν το μοντέλο ότι δεν επιτρέπεται να δημιουργεί πληροφορίες που δεν υπάρχουν στη βάση γνώσης.

Για παράδειγμα, το μοντέλο δεν πρέπει να εφευρίσκει:

- ονόματα εστιατορίων,
- ώρες λειτουργίας,
- τιμές,
- διαθεσιμότητα,
- κρατήσεις,
- προγράμματα λειτουργίας.

Αν οι πληροφορίες δεν υπάρχουν στη βάση γνώσης, η εφαρμογή ενημερώνει τον επισκέπτη ότι πρέπει να επικοινωνήσει με το Concierge για επιβεβαίωση.

8.5 Ένα πραγματικό παράδειγμα από την ανάπτυξη

Κατά τη διάρκεια των δοκιμών διαπίστωσα ότι όταν ρωτούσα:

What are the Spa opening hours?

το μοντέλο κάποιες φορές απαντούσε με διαφορετικό ωράριο από αυτό που υπήρχε στη βάση γνώσης. Για να λύσω αυτό το πρόβλημα:

- πρόσθεσα πιο αυστηρούς Grounding Rules,
- βελτίωσα το Retrieval,
- εμπλούτισα τα αρχεία Markdown με πιο σαφείς πληροφορίες.

Μετά τις αλλαγές αυτές, οι απαντήσεις έγιναν πολύ πιο σταθερές και πιο αξιόπιστες.

9. Retrieval-Augmented Generation (RAG)

Η σημαντικότερη τεχνική που χρησιμοποιεί η ESTIA είναι το **Retrieval-Augmented Generation (RAG)**. Με απλά λόγια, το μοντέλο δεν προσπαθεί να θυμηθεί τις πληροφορίες του ξενοδοχείου. Αντίθετα, κάθε φορά που ο επισκέπτης κάνει μία ερώτηση, η εφαρμογή ψάχνει πρώτα στη δική της βάση γνώσης. Μόνο αφού βρει τα πιο σχετικά έγγραφα, τα στέλνει μαζί με την ερώτηση στο GPT-4o. Αυτό σημαίνει ότι η γνώση της εφαρμογής μπορεί να ενημερώνεται οποιαδήποτε στιγμή, χωρίς να χρειάζεται επανεκπαίδευση του μοντέλου.

Θεωρώ ότι αυτή ήταν μία από τις σημαντικότερες επιλογές της αρχιτεκτονικής, γιατί κάνει την εφαρμογή πιο αξιόπιστη και πολύ πιο εύκολη στη συντήρηση.

10. Δομή του κώδικα της εφαρμογής

Κατά την ανάπτυξη της ESTIA προσπάθησα να οργανώσω τον κώδικα σε μικρά και ανεξάρτητα τμήματα. Στόχος μου ήταν κάθε αρχείο να έχει μία συγκεκριμένη ευθύνη και να μπορεί να τροποποιηθεί χωρίς να επηρεάζει ολόκληρη την εφαρμογή.

Η βασική δομή του project είναι η παρακάτω.

app/

└─ api/

└─ core/

└─ models/

└─ services/

└─ scripts/

└─ utils/

Στις επόμενες ενότητες παρουσιάζω συνοπτικά τον ρόλο κάθε φακέλου.

10.1 Ο φάκελος **app**

Ο φάκελος **app** περιέχει όλη τη λογική της εφαρμογής. Μπορούμε να πούμε ότι αποτελεί τον βασικό κορμό της ESTIA.

Μέσα σε αυτόν υπάρχουν οι φάκελοι:

- api
- core
- models
- services
- scripts
- utils

Καθένας από αυτούς έχει διαφορετικό ρόλο.

10.2 Ο φάκελος **api**

Ο φάκελος **api** περιέχει όλα τα REST API της εφαρμογής. Είναι το σημείο από το οποίο επικοινωνεί το frontend με το backend. Στην εφαρμογή υπάρχουν τα παρακάτω βασικά API.

chat.py

Το σημαντικότερο αρχείο είναι το **chat.py**. Κάθε φορά που ο επισκέπτης στέλνει μία ερώτηση, αυτή καταλήγει αρχικά στο συγκεκριμένο αρχείο. Εδώ πραγματοποιούνται τα πρώτα βήματα της επεξεργασίας.

Συγκεκριμένα:

- αναγνωρίζεται η γλώσσα,
- ενημερώνεται η μνήμη του επισκέπτη,
- δημιουργείται το κατάλληλο retrieval query,
- αναζητούνται πληροφορίες στη βάση γνώσης,
- αποστέλλεται το τελικό prompt στο GPT.

Θεωρώ ότι το συγκεκριμένο αρχείο είναι ο "συντονιστής" ολόκληρης της εφαρμογής.

health.py

Το αρχείο **health.py** χρησιμοποιείται για τον έλεγχο της κατάστασης της εφαρμογής. Επιστρέφει μία απλή απάντηση που δείχνει ότι ο server λειτουργεί σωστά. Χρησιμοποιείται κυρίως κατά την ανάπτυξη και τις δοκιμές.

image.py

Το συγκεκριμένο endpoint δημιουργήθηκε ώστε στο μέλλον η ESTIA να μπορεί να υποστηρίζει εικόνες. Παρότι στην παρούσα έκδοση δεν χρησιμοποιείται πλήρως, αποτέλεσε μέρος του αρχικού σχεδιασμού.

voice.py

Αντίστοιχα, το endpoint **voice.py** δημιουργήθηκε ώστε η εφαρμογή να μπορεί μελλοντικά να υποστηρίζει φωνητική συνομιλία. Με αυτόν τον τρόπο η αρχιτεκτονική είναι ήδη έτοιμη για μελλοντικές επεκτάσεις.

10.3 Ο φάκελος services

Ο φάκελος **services** είναι ίσως ο σημαντικότερος της εφαρμογής. Εδώ βρίσκεται όλη η επιχειρησιακή λογική (Business Logic). Αν αφαιρέσουμε τον φάκελο αυτό, η εφαρμογή δεν μπορεί να λειτουργήσει.

chat_service.py

Το αρχείο αυτό είναι υπεύθυνο για τη δημιουργία του τελικού prompt.

Συγκεντρώνει:

- το system prompt,
- τις πληροφορίες από τη βάση γνώσης,
- το Guest Context,
- το ιστορικό της συνομιλίας.

Στη συνέχεια στέλνει όλα αυτά στο GPT-4o και λαμβάνει την τελική απάντηση.

Μπορούμε να πούμε ότι αποτελεί το σημείο όπου ενώνονται όλα τα επιμέρους τμήματα της εφαρμογής.

rag_service.py

Το συγκεκριμένο αρχείο είναι υπεύθυνο για την αναζήτηση πληροφοριών στη βάση γνώσης. Χρησιμοποιεί τη ChromaDB για να εντοπίσει τα πιο σχετικά έγγραφα. Δεν επιστρέφει ολόκληρα αρχεία αλλά μόνο τα πιο σχετικά αποσπάσματα. Έτσι μειώνεται ο αριθμός των tokens που αποστέλλονται στο GPT.

language_service.py

To language service δημιουργεί τα system prompts.

Η ESTIA υποστηρίζει δύο γλώσσες:

- Ελληνικά
- Αγγλικά

Για κάθε γλώσσα χρησιμοποιείται διαφορετικό prompt. Στο συγκεκριμένο αρχείο πρόσθεσα και τους κανόνες Grounding ώστε το μοντέλο να μην δημιουργεί πληροφορίες που δεν υπάρχουν στη βάση γνώσης.

context_extraction.py

Το αρχείο αυτό δημιουργήθηκε κατά τη διάρκεια της ανάπτυξης του Guest Context Memory. Σκοπός του είναι να αναγνωρίζει πληροφορίες μέσα από το μήνυμα του επισκέπτη.

Για παράδειγμα:

We stay at Porto Elounda.

Η εφαρμογή καταλαβαίνει ότι:

Property = Porto Elounda

Αν ο επισκέπτης γράψει:

We have children aged 5 and 8.

η εφαρμογή αποθηκεύει ότι πρόκειται για οικογένεια με δύο παιδιά.

Το συγκεκριμένο αρχείο δεν χρησιμοποιεί Τεχνητή Νοημοσύνη. Βασίζεται σε κανόνες και λέξεις-κλειδιά, κάτι που το κάνει πιο γρήγορο και πιο εύκολο στον έλεγχο.

retrieval_context.py

Το retrieval_context δημιουργήθηκε σε μεταγενέστερο στάδιο. Ο ρόλος του είναι να βελτιώνει την αναζήτηση στη βάση γνώσης. Αντί να στέλνει μόνο την ερώτηση του επισκέπτη, δημιουργεί μία πιο πλήρη περιγραφή.

Για παράδειγμα:

Αν ο επισκέπτης γράψει:

Where should we have dinner?

η εφαρμογή γνωρίζει ήδη ότι:

- μένει στο Porto Elounda,
- είναι οικογένεια,
- έχει δύο παιδιά.

Έτσι δημιουργείται ένα πιο ολοκληρωμένο retrieval query. Αυτό αυξάνει σημαντικά την ποιότητα των αποτελεσμάτων.

conversation_store.py

Το συγκεκριμένο αρχείο αποθηκεύει:

- το ιστορικό της συνομιλίας,
- το Guest Context.

Κάθε συνομιλία έχει το δικό της μοναδικό Conversation ID. Έτσι διαφορετικοί χρήστες δεν επηρεάζουν ο ένας τον άλλον.

voice_service.py

Το voice service αποτελεί τη βάση για τη μελλοντική φωνητική λειτουργία της εφαρμογής. Στην παρούσα εργασία η λειτουργία αυτή βρίσκεται ακόμη σε αρχικό στάδιο.

image_service.py

Αντίστοιχα, το image service δημιουργήθηκε ώστε η ESTIA να μπορεί στο μέλλον να επεξεργάζεται εικόνες. Η αρχιτεκτονική της εφαρμογής έχει σχεδιαστεί έτσι ώστε νέες δυνατότητες να μπορούν να προστεθούν χωρίς να αλλάξει η υπόλοιπη εφαρμογή.

10.4 Ο φάκελος models

Στον φάκελο **models** βρίσκονται τα μοντέλα δεδομένων της εφαρμογής. Τα μοντέλα αυτά χρησιμοποιούνται για την ανταλλαγή πληροφοριών μεταξύ των διαφόρων τμημάτων της ESTIA.

Για παράδειγμα:

- ChatRequest
- ChatResponse

- GuestContext

Το σημαντικότερο μοντέλο που δημιούργησα είναι το **GuestContext**, επειδή επιτρέπει στην εφαρμογή να θυμάται πληροφορίες κατά τη διάρκεια της συνομιλίας.

10.5 Ο φάκελος **core**

Ο φάκελος **core** περιέχει βασικές ρυθμίσεις της εφαρμογής.

Εδώ βρίσκονται:

- οι μεταβλητές περιβάλλοντος,
- οι ρυθμίσεις,
- το logging.

Με αυτόν τον τρόπο όλες οι βασικές παράμετροι βρίσκονται συγκεντρωμένες σε ένα σημείο.

10.6 Ο φάκελος **scripts**

Ο φάκελος **scripts** περιέχει βοηθητικά προγράμματα. Το σημαντικότερο είναι το πρόγραμμα που διαβάζει όλα τα αρχεία Markdown και τα εισάγει στη βάση ChromaDB. Η διαδικασία αυτή ονομάζεται **ingestion**. Χωρίς αυτό το βήμα η εφαρμογή δεν θα μπορούσε να πραγματοποιήσει αναζήτηση στη βάση γνώσης.

10.7 Ο φάκελος **utils**

Τέλος, ο φάκελος **utils** περιέχει μικρές βοηθητικές συναρτήσεις που χρησιμοποιούνται σε διάφορα σημεία του project.

Η ύπαρξη αυτού του φακέλου βοηθά ώστε να μην επαναλαμβάνεται ο ίδιος κώδικας σε πολλά αρχεία.

11. Πώς συνεργάζονται όλα μεταξύ τους

Στην αρχή της ανάπτυξης θεωρούσα ότι η εφαρμογή αποτελείται από ανεξάρτητα αρχεία. Στην πραγματικότητα όμως όλα συνεργάζονται μεταξύ τους σαν μία αλυσίδα.

Η συνολική διαδικασία είναι η παρακάτω.

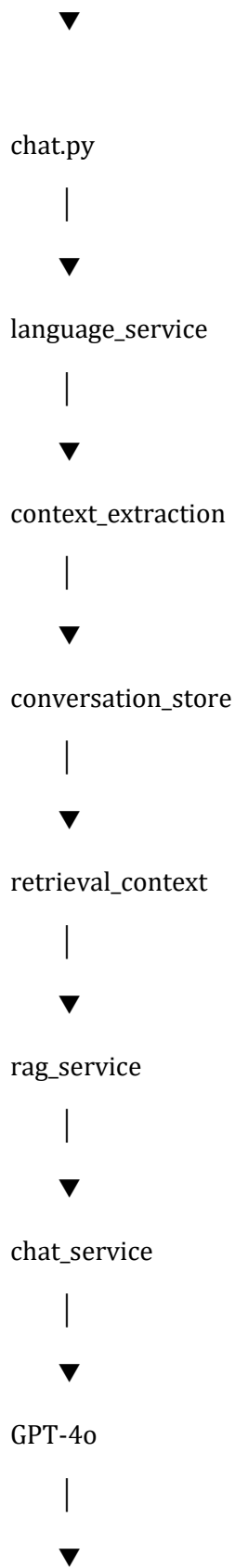
Ο επισκέπτης γράφει μία ερώτηση

|



Frontend

|



Απάντηση στον επισκέπτη

Με αυτή τη σχεδίαση κάθε αρχείο έχει μία συγκεκριμένη ευθύνη. Αν στο μέλλον χρειαστεί να αλλάξω κάποιο τμήμα της εφαρμογής, μπορώ να το κάνω χωρίς να επηρεαστεί ολόκληρο το σύστημα.

Αυτό θεωρώ ότι είναι ένα από τα σημαντικότερα πλεονεκτήματα της αρχιτεκτονικής που επέλεξα.

12. Οι δυσκολίες που αντιμετώπισα κατά την ανάπτυξη

Κατά την ανάπτυξη της εφαρμογής συνάντησα αρκετές δυσκολίες. Οι περισσότερες δεν σχετίζονταν με την Python, αλλά με τον τρόπο που λειτουργούν τα μεγάλα γλωσσικά μοντέλα (Large Language Models). Μέσα από αυτές τις δυσκολίες κατάλαβα ότι η ανάπτυξη μιας εφαρμογής τεχνητής νοημοσύνης δεν είναι μόνο θέμα προγραμματισμού. Απαιτεί σωστό σχεδιασμό της αρχιτεκτονικής και καλή κατανόηση του τρόπου με τον οποίο συνεργάζονται όλα τα επιμέρους τμήματα.

Παρακάτω παρουσιάζω τα σημαντικότερα προβλήματα που αντιμετώπισα και τον τρόπο με τον οποίο προσπάθησα να τα επιλύσω.

12.1 Hallucinations

Το πρώτο πρόβλημα που συνάντησα ήταν τα λεγόμενα **Hallucinations**. Αυτό σημαίνει ότι το μοντέλο δημιουργούσε πληροφορίες οι οποίες δεν υπήρχαν στην πραγματικότητα. Για παράδειγμα, όταν ρωτούσα για κάποιο εστιατόριο, υπήρχαν περιπτώσεις όπου το GPT δημιουργούσε ονόματα εστιατορίων που δεν υπήρχαν στο ξενοδοχείο.

Αντίστοιχα μπορούσε να δημιουργήσει:

- λάθος ώρες λειτουργίας,
- λάθος τιμές,
- υπηρεσίες που δεν παρέχονταν,
- δραστηριότητες που δεν υπήρχαν.

Για μία εφαρμογή που προορίζεται να εξυπηρετεί πραγματικούς επισκέπτες, κάτι τέτοιο αποτελεί σοβαρό πρόβλημα. Για να το αντιμετωπίσω, αποφάσισα να χρησιμοποιήσω την τεχνική Retrieval-Augmented Generation (RAG), ώστε το μοντέλο να βασίζεται στις πληροφορίες της δικής μου βάσης γνώσης.

12.2 Η εφαρμογή δεν θυμόταν τον επισκέπτη

Ένα δεύτερο πρόβλημα εμφανίστηκε όταν δοκίμαζα μεγαλύτερες συνομιλίες.

Για παράδειγμα:

We stay at Porto Elounda.

και λίγα λεπτά αργότερα:

Where should we have dinner?

Σε αρκετές περιπτώσεις η εφαρμογή είχε ήδη "ξεχάσει" σε ποιο ξενοδοχείο βρισκόταν ο επισκέπτης. Το αποτέλεσμα ήταν να προτείνει εστιατόρια από διαφορετικές μονάδες. Αυτό με οδήγησε στη δημιουργία του **Guest Context Memory**, το οποίο αποθηκεύει τις βασικές πληροφορίες του επισκέπτη και τις χρησιμοποιεί σε ολόκληρη τη συνομιλία.

12.3 Μη σχετικές απαντήσεις

Ένα ακόμη πρόβλημα ήταν ότι η εφαρμογή έβρισκε σωστά έγγραφα, αλλά όχι πάντα τα πιο σχετικά. Για παράδειγμα, όταν ο επισκέπτης ρωτούσε:

Where should we have dinner?

η εφαρμογή μπορούσε να επιστρέψει γενικές πληροφορίες σχετικά με το ξενοδοχείο αντί για πληροφορίες σχετικά με τα εστιατόρια.

Για να βελτιώσω την αναζήτηση δημιούργησα το **Intent Detection**. Με αυτόν τον τρόπο η εφαρμογή προσπαθεί πρώτα να καταλάβει το θέμα της ερώτησης και στη συνέχεια πραγματοποιεί πιο στοχευμένη αναζήτηση.

12.4 Grounding Rules

Παρόλο που η εφαρμογή χρησιμοποιούσε ήδη RAG, παρατήρησα ότι το GPT μπορούσε ακόμη να συμπληρώνει πληροφορίες που δεν υπήρχαν στα αρχεία Markdown.

Για παράδειγμα:

- να υποθέσει ώρες λειτουργίας,
- να υποθέσει τιμές,
- να υποθέσει διαθεσιμότητα.

Για τον λόγο αυτό πρόσθεσα ειδικούς κανόνες στο System Prompt. Οι κανόνες αυτοί ενημερώνουν το μοντέλο ότι επιτρέπεται να χρησιμοποιεί μόνο τις πληροφορίες που βρίσκονται στη βάση γνώσης. Αν δεν υπάρχουν διαθέσιμα δεδομένα, η εφαρμογή προτείνει στον επισκέπτη να επικοινωνήσει με το Concierge. Μετά από αυτή την αλλαγή οι απαντήσεις έγιναν αισθητά πιο αξιόπιστες.

13. Δοκιμές της εφαρμογής

Καθ' όλη τη διάρκεια της ανάπτυξης πραγματοποιούσα συνεχώς δοκιμές. Δεν περίμενα να ολοκληρωθεί η εφαρμογή για να ελέγξω αν λειτουργεί σωστά. Αντίθετα, μετά από κάθε νέα

λειτουργία πραγματοποιούσα νέα tests. Αυτό με βοήθησε να εντοπίζω προβλήματα αρκετά νωρίς.

13.1 Unit Tests

Αρχικά δημιούργησα Unit Tests. Τα tests αυτά ελέγχουν μικρά τμήματα του κώδικα ανεξάρτητα από τα υπόλοιπα.

Για παράδειγμα:

- Context Extraction
- Conversation Store
- Retrieval Context
- Guest Context
- Intent Detection

Με αυτόν τον τρόπο μπορούσα να γνωρίζω ότι κάθε λειτουργία δούλεψε σωστά πριν προχωρήσω στην επόμενη.

13.2 Integration Tests

Στη συνέχεια δημιούργησα Integration Tests. Τα tests αυτά ελέγχουν αν όλα τα τμήματα της εφαρμογής συνεργάζονται σωστά μεταξύ τους. Για παράδειγμα:

Guest Context

↓

Retrieval

↓

GPT

↓

Response

Αυτό είναι πολύ σημαντικό γιατί μία λειτουργία μπορεί να δουλεύει σωστά μόνη της αλλά να παρουσιάζει προβλήματα όταν συνεργάζεται με τις υπόλοιπες.

13.3 Πραγματικά σενάρια

Εκτός από τα αυτοματοποιημένα tests δημιούργησα και πραγματικά σενάρια συνομιλίας.

Παραδείγματα:

Οικογένεια με δύο παιδιά.

Ζευγάρι σε ταξίδι του μέλιτος.

VIP επισκέπτης.

Μεταφορά από το αεροδρόμιο.

Κράτηση Yacht.

Ερωτήσεις για το Spa.

Ερωτήσεις στα Ελληνικά.

Ερωτήσεις στα Αγγλικά.

Τα σενάρια αυτά με βοήθησαν να αξιολογήσω την εφαρμογή όπως θα τη χρησιμοποιούσε ένας πραγματικός επισκέπτης.

14. Αποτελέσματα

Μετά την ολοκλήρωση της ανάπτυξης η εφαρμογή μπορούσε να:

- απαντά στα Ελληνικά και στα Αγγλικά,
- αναζητά πληροφορίες στη βάση γνώσης,
- θυμάται βασικά στοιχεία του επισκέπτη,
- προσαρμόζει τις απαντήσεις της,
- περιορίζει σημαντικά τα hallucinations,
- χρησιμοποιεί πραγματικά δεδομένα του ξενοδοχείου,
- οργανώνει τη γνώση σε δομημένη μορφή.

Πιστεύω ότι η ESTIA αποτελεί μία καλή βάση για την ανάπτυξη ενός πραγματικού AI Concierge.

Παρόλο που υπάρχουν ακόμη δυνατότητες βελτίωσης, η εφαρμογή κατάφερε να καλύψει όλους τους αρχικούς στόχους που είχα θέσει κατά την έναρξη της εργασίας.

15. Μελλοντικές Δυνατότητες (Future Improvements)

Παρόλο που η εφαρμογή ESTIA καλύπτει τους αρχικούς στόχους που είχα θέσει, θεωρώ ότι υπάρχουν αρκετές δυνατότητες επέκτασης. Κατά τη διάρκεια της ανάπτυξης προέκυψαν πολλές ιδέες, οι οποίες δεν υλοποιήθηκαν είτε λόγω χρονικών περιορισμών είτε επειδή απαιτούν πρόσβαση σε πραγματικά πληροφοριακά συστήματα ξενοδοχείων.

Στη συνέχεια παρουσιάζω τις σημαντικότερες από αυτές.

15.1 Διασύνδεση με Opera PMS

Στην παρούσα έκδοση η ESTIA δεν έχει πρόσβαση σε πραγματικά στοιχεία κρατήσεων. Μία σημαντική βελτίωση θα ήταν η σύνδεση με το Opera PMS (Property Management System), το οποίο χρησιμοποιείται από πολλά ξενοδοχεία.

Με αυτόν τον τρόπο η εφαρμογή θα μπορούσε να γνωρίζει:

- το όνομα του επισκέπτη,
- τον αριθμό δωματίου,
- τις ημερομηνίες άφιξης και αναχώρησης,
- τον τύπο δωματίου,
- το πρόγραμμα διατροφής.

Έτσι οι απαντήσεις θα ήταν ακόμη πιο προσωποποιημένες.

15.2 Online Κρατήσεις

Σήμερα η εφαρμογή ενημερώνει τον επισκέπτη σχετικά με τις υπηρεσίες του ξενοδοχείου. Στο μέλλον θα μπορούσε να πραγματοποιεί και πραγματικές κρατήσεις.

Για παράδειγμα:

- κράτηση εστιατορίου,
- κράτηση Spa,
- κράτηση γηπέδου Golf,
- κράτηση Yacht,
- κράτηση μεταφοράς από το αεροδρόμιο.

Με αυτόν τον τρόπο η ESTIA θα λειτουργούσε όχι μόνο ως σύμβουλος αλλά και ως ψηφιακός concierge.

15.3 Φωνητική Συνομιλία

Η εφαρμογή έχει ήδη σχεδιαστεί ώστε να μπορεί να υποστηρίξει φωνητική επικοινωνία. Μία μελλοντική έκδοση θα μπορούσε να επιτρέπει στον επισκέπτη να συνομιλεί με φυσική φωνή, χωρίς να χρειάζεται να πληκτρολογεί. Η δυνατότητα αυτή θα ήταν ιδιαίτερα χρήσιμη για ηλικιωμένους επισκέπτες ή για άτομα που προτιμούν τη φωνητική επικοινωνία.

15.4 Mobile Application

Η ESTIA λειτουργεί σήμερα μέσω web browser. Μία φυσική συνέχεια του έργου θα ήταν η ανάπτυξη εφαρμογής για κινητές συσκευές Android και iOS. Με αυτόν τον τρόπο ο επισκέπτης θα μπορούσε να χρησιμοποιεί την εφαρμογή οπουδήποτε μέσα στο ξενοδοχείο.

15.5 Υποστήριξη περισσότερων ξενοδοχείων

Η εφαρμογή σχεδιάστηκε εξ αρχής με τρόπο ώστε να μπορεί να υποστηρίξει περισσότερες από μία ξενοδοχειακές μονάδες. Στο μέλλον θα μπορούσε να χρησιμοποιηθεί από ολόκληρες ξενοδοχειακές αλυσίδες, με ξεχωριστή βάση γνώσης για κάθε ξενοδοχείο. Η αρχιτεκτονική που ακολουθήθηκε επιτρέπει αυτή την επέκταση χωρίς σημαντικές αλλαγές στον κώδικα.

15.6 Διασύνδεση με CRM

Μία ακόμη δυνατότητα είναι η σύνδεση της ESTIA με κάποιο σύστημα CRM. Έτσι η εφαρμογή θα μπορούσε να γνωρίζει το ιστορικό του επισκέπτη, τις προηγούμενες κρατήσεις και τις προσωπικές του προτιμήσεις. Οι απαντήσεις θα ήταν ακόμη πιο προσωποποιημένες.

15.7 Συνεχής Ενημέρωση της Βάσης Γνώσης

Στην παρούσα έκδοση η ενημέρωση της γνώσης γίνεται χειροκίνητα. Μία μελλοντική επέκταση θα μπορούσε να επιτρέπει την αυτόματη ενημέρωση της βάσης γνώσης από τα πληροφοριακά συστήματα του ξενοδοχείου. Έτσι οποιαδήποτε αλλαγή στα εστιατόρια, στις ώρες λειτουργίας ή στις υπηρεσίες θα εμφανιζόταν αυτόματα και στην εφαρμογή.

15.8 Προσωπική Αξιολόγηση

Η ανάπτυξη της ESTIA μου έδειξε ότι οι εφαρμογές τεχνητής νοημοσύνης μπορούν να αξιοποιηθούν ουσιαστικά στον χώρο της φιλοξενίας. Παρόλο που η εφαρμογή αποτελεί πρωτότυπο (prototype), θεωρώ ότι η αρχιτεκτονική της είναι αρκετά ευέλικτη ώστε να αποτελέσει τη βάση για μία πραγματική παραγωγική εφαρμογή. Στο μέλλον θα ήθελα να συνεχίσω την ανάπτυξή της και να προσθέσω περισσότερες λειτουργίες, αξιοποιώντας τις γνώσεις που απέκτησα μέσα από αυτή την εργασία.

16. Τι έμαθα μέσα από αυτή την εργασία

Η ανάπτυξη της ESTIA ήταν το μεγαλύτερο έργο Python και Τεχνητής Νοημοσύνης που έχω υλοποιήσει μέχρι σήμερα. Μέσα από την εργασία αυτή κατάλαβα ότι η δημιουργία μίας εφαρμογής AI δεν περιορίζεται στην επικοινωνία με ένα γλωσσικό μοντέλο. Αντίθετα, απαιτεί σωστή οργάνωση της γνώσης, κατάλληλη αρχιτεκτονική και συνεχείς δοκιμές.

Έμαθα να χρησιμοποιώ εργαλεία όπως:

- Python,
- FastAPI,

- OpenAI API,
- ChromaDB,
- Git,
- GitHub,
- Markdown,
- Pytest.

Παράλληλα, εξοικειώθηκα με έννοιες που μέχρι πρόσφατα γνώριζα μόνο θεωρητικά, όπως το Retrieval-Augmented Generation, οι Vector Databases και το Prompt Engineering. Θεωρώ ότι η συγκεκριμένη εργασία με βοήθησε να συνδέσω τη θεωρία με την πράξη και να αποκτήσω μεγαλύτερη αυτοπεποίθηση στην ανάπτυξη εφαρμογών τεχνητής νοημοσύνης.

17. Συμπεράσματα

Στόχος της εργασίας ήταν η ανάπτυξη ενός βοηθού τεχνητής νοημοσύνης για ξενοδοχειακές μονάδες, ο οποίος να μπορεί να απαντά σε πραγματικές ερωτήσεις επισκεπτών χρησιμοποιώντας αξιόπιστες πληροφορίες. Κατά τη διάρκεια της ανάπτυξης υλοποίησα μία εφαρμογή βασισμένη στην Python και στο FastAPI, η οποία αξιοποιεί το GPT-4o σε συνδυασμό με μία οργανωμένη βάση γνώσης και την τεχνική Retrieval-Augmented Generation. Παράλληλα, πρόσθεσα μηχανισμούς όπως το Guest Context Memory και το Grounded Retrieval ώστε οι απαντήσεις να είναι πιο προσωποποιημένες και πιο αξιόπιστες.

Πιστεύω ότι η εργασία αυτή αποτελεί μία ολοκληρωμένη εφαρμογή, η οποία θα μπορούσε να εξελιχθεί ακόμη περισσότερο με την προσθήκη νέων λειτουργιών, όπως σύνδεση με συστήματα κρατήσεων, φωνητική επικοινωνία και πρόσβαση σε πραγματικά δεδομένα του ξενοδοχείου.

Κλείνοντας, θεωρώ ότι μέσα από την ανάπτυξη της ESTIA απέκτησα πολύτιμη εμπειρία τόσο στον προγραμματισμό όσο και στον σχεδιασμό εφαρμογών τεχνητής νοημοσύνης και κατάλαβα ότι η σωστή αρχιτεκτονική είναι εξίσου σημαντική με τον ίδιο τον κώδικα.

Παράρτημα

Το παράρτημα περιλαμβάνει συνοπτικά τη δομή του έργου και βασικές πληροφορίες που διευκολύνουν την κατανόηση της εφαρμογής.

Παράρτημα Α – Δομή του Project

estia-ai-concierge/



Παράρτημα Β – Βασικά Endpoints

Endpoint	Περιγραφή
/api/v1/chat	Επεξεργάζεται τα μηνύματα του χρήστη και επιστρέφει την απάντηση της ESTIA.
/api/v1/health	Ελέγχει αν η εφαρμογή λειτουργεί σωστά.
/api/v1/image	Endpoint για μελλοντική υποστήριξη εικόνων.
/api/v1/voice	Endpoint για μελλοντική φωνητική επικοινωνία.

Παράρτημα Γ – Βασικές Τεχνολογίες

Τεχνολογία	Χρήση
Python 3.13	Ανάπτυξη της εφαρμογής
FastAPI	Backend και REST API
OpenAI GPT-4o	Δημιουργία απαντήσεων
ChromaDB	Vector Database
Markdown	Βάση γνώσης
HTML / CSS / JavaScript	Γραφικό περιβάλλον
Pytest	Έλεγχος λειτουργίας
Git / GitHub	Διαχείριση εκδόσεων

Παράρτημα Δ – GitHub

Ο πηγαίος κώδικας της εφαρμογής φιλοξενείται στο GitHub και περιλαμβάνει:

- τον πλήρη κώδικα της εφαρμογής,
- τη βάση γνώσης,
- τα αυτοματοποιημένα tests,
- τα screenshots,
- το README του έργου,
- το ιστορικό ανάπτυξης μέσω Git.